

Tema 06: Asincronía y comunicación con el servidor

RA 07

Parte 01

Ejercicios

Antes de empezar:

1. Dentro del repositorio dwec_ejercicios_apellido1_nombre1_25-26

Usa la plantilla para crear la carpeta:

- dwec_**t06p01**_apellido1_nombre1

Ejercicio 1.

Crea una carpeta con nombre **ejercicio1** dentro de dwec_**t06p01**_apellido1_nombre1.

En esta práctica vamos a crear una aplicación web que consuma una API externa usando AJAX moderno, concretamente la fetch junto con async / await.

La aplicación estará basada en la API pública de Harry Potter, que devuelve información en formato JSON: <https://hp-api.onrender.com>

Será necesario crear un sitio web de UNA solo página (con varias secciones) haciendo uso de Bootstrap via CDN y su validación de formularios.

También se tiene que incluir una estructura básica de página formada por:

- header con clases de Bootstrap
- navbar que nos permita navegar entre las distintas secciones de la página
- main como contenedor principal
- footer también usando estilos de Bootstrap

El sitio web será adaptativo al menos para un corte móvil y un corte de escritorio.



El sitio web constará de las siguientes secciones, todas visibles en la misma página y todas funcionarán con peticiones ASÍNCRONAS.

Se utilizará la API pública de Harry Potter: <https://hp-api.onrender.com/> La API devuelve datos en formato JSON y no requiere autenticación.

Muy importante: En una aplicación que consume una API externa no todo va bien siempre. La aplicación deberá gestionar correctamente los errores derivados del consumo de la API, diferenciando entre:

- Errores de red (fetch lanza una excepción)
- Errores HTTP (la API responde, pero con error: 404, 500, etc.)
- Búsquedas sin resultados (la API devuelve un array vacío)
- Errores de uso por parte del usuario (input vacío)

En todos los casos se deberá mostrar un mensaje adecuado en la interfaz.

Sección 1: Buscador

Esta sección permitirá buscar elementos (personajes) en la API de Harry Potter. Para ello habrá:

- Campo de texto para introducir un criterio de búsqueda (por ejemplo, nombre del personaje).
- Botón de búsqueda.

Al pulsar el botón **o a medida que se va escribiendo**:

- Se realizará una petición AJAX a la API.
- Se mostrarán los resultados obtenidos dinámicamente en la página.
- Los resultados se mostrarán usando una tabla con la información más importante y una imagen en miniatura. La información de la tabla se hará creando **los nodos del DOM**.
- Cada fila tendrá un botón llamado “Marcar como favorito”.
 - El botón Marcar como favorito no tendrá funcionalidad en este ejercicio. **Se implementará en un ejercicio posterior.**



- Si no hay resultados de búsqueda se informará de ello.

Sección 2: Bienvenida

Esta sección se cargará automáticamente al iniciar la página. Se obtendrán ocho personajes aleatorios de la API, dos personajes de cada casa. Cada personaje se mostrará en una tarjeta Bootstrap. Las tarjetas se dibujaran usando Template Strings. Cada tarjeta deberá mostrar obligatoriamente:

- Foto del personaje
- Nombre
- Casa
- Patronus
- Especie
- Año de nacimiento

Será necesario mostrar un loader/spinner mientras se cargan los datos. Para ello, simular una espera con `setTimeout` antes de mostrar los resultados.

Nota: Esta API no permite buscar por nombre, así que será necesario conseguir todos los personajes y filtrarlos con JS. Aquí tenemos dos formas válidas de proceder:

1. Pedir todos los datos a la API en el `DOMContentLoaded` y usar una variable global para almacenar todos los personajes: `let todosLosPersonajes = []`; Después se filtra ese array.
2. Cada vez que haya una búsqueda se piden todos los datos a la API y cuando llegan se filtran.

Sección 3: Favoritos

Esta sección mostrará los personajes marcados como favoritos. En este ejercicio No se implementará ninguna funcionalidad real. Se mostrará un texto del tipo: “La gestión de favoritos se implementará en breve.”

Sección 4: Quiénes somos

Esta sección será informativa. Contendrá texto estático explicando el propósito de la aplicación.
De momento no tendrá funcionalidad JavaScript en este ejercicio.

Ejercicio 2.

Amplía el ejercicio 1 con nuevas funcionalidades basadas en APIs Web del navegador.

Funcionalidades a implementar:

Función 1. Aceptación de cookies (almacenamiento de sesión)

Al acceder a la aplicación por primera vez, se deberá mostrar un aviso de cookies. El aviso se mostrará solo si el usuario no ha aceptado previamente. Al aceptar las cookies:

- El aviso desaparecerá.
- La aceptación se guardará usando la Web Storage API: sessionStorage
 - No se usarán cookies reales en este ejercicio

Nota: sessionStorage mantiene los datos mientras dura la sesión del navegador.

Función 2. Geolocalización y mapa del IES

Se hará la sección en “Quiénes somos” para mostrar la ubicación del centro educativo en un mapa.
Para ello:

- Mostrar un mapa con la ubicación del IES (posición fija).
- Mostrar opcionalmente, la ubicación del usuario.
 - Usa la Web Geolocation API para obtener la ubicación del usuario (si acepta).
 - Muestra un mensaje adecuado si el usuario no permite la geolocalización.

Puedes elegir Google Maps o Leaflet + OpenStreetMap:

- Google Maps: requiere cuenta y clave API y tiene límites y condiciones
- Leaflet: es una librería ligera de mapas. Se recomienda usar esta.

Función 3. Sistema de favoritos con localStorage

En este ejercicio se completa la funcionalidad de Favoritos. Para ello:

- Al pulsar el botón “Marcar como favorito”:
 - El personaje se guardará en localStorage porque los favoritos deben persistir al recargar la página. Se almacenarán los personajes como objetos JavaScript convertidos a JSON con JSON.stringify()
 - Si el personaje ya está en favoritos. el botón permitirá eliminarlo.

La sección Favoritos deberá mostrar los personajes guardados. Para mostrar los datos se hará uso de List Group (creando nodos o con String templates):
<https://getbootstrap.com/docs/4.0/components/list-group/#custom-content>

Nota: observa la diferencia en localStorage y sessionStorage.